Southeast University

Department of Computer Science and Engineering

Course Name: Database Design Lab

Course Code: CSE384

Lab No: 10

Lab Title: Exploring SQL JOINs

Date of Submission: 12/01/2026

Submitted To:

S I M Adnan

Lecturer, Department of CSE, Southeast University

Initial: IN

Submitted By:

Name: Nazmul Hasan

ID: 2023100000130

Course: CSE384

Section: 07

Objective: Learn to combine data from multiple tables using different types of JOINs: INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.

Problems and Solutions:

Problem 1:

Title: Find all employees with their department names and salaries using an INNER JOIN.

SQL Query:
```
SELECT e.name, e.salary, d.dept_name FROM Employee e INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

Output:

Showing rows 0 - 5 (6 total, Query took 0.0004 seconds.)

```
SELECT e.name, e.salary, d.dept_name FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

name	salary	dept_name
Arif	45000	HR
Nila	60000	IT
Sumon	55000	Finance
Rina	65000	IT
Karim	*NULL*	Marketing
Jamil	50000	Finance

Problem 2:

Title : Find all employees and their department names, including employees with no department using LEFT JOIN.

SQL Query:
```
SELECT e.name, e.salary, d.dept_name FROM Employee e LEFT JOIN Department d
ON e.dept_id = d.dept_id;
```

Output:

Showing rows 0 - 5 (6 total, Query took 0.0018 seconds.)

```
SELECT e.name, e.salary, d.dept_name FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

name	salary	dept_name
Arif	45000	HR
Nila	60000	IT
Sumon	55000	Finance
Rina	65000	IT
Karim	NULL	Marketing
Jamil	50000	Finance

Problem 3:

Title: Find all departments and their employees, including departments with no employees using RIGHT JOIN.

SQL Query:

```
SELECT e.name, e.salary, d.dept_name FROM Employee e RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

Output:

Showing rows 0 - 5 (6 total, Query took 0.0019 seconds.)

```
SELECT e.name, e.salary, d.dept_name FROM Employee e RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

name	salary	dept_name
Arif	45000	HR
Nila	60000	IT
Rina	65000	IT
Sumon	55000	Finance
Jamil	50000	Finance
Karim	NULL	Marketing

Problem 4:

Title: Find all employees and departments, showing all matches and non-matches using FULL JOIN.

SQL Query:

```
SELECT e.name, e.salary, d.dept_name
FROM Employee e
FULL JOIN Department d ON e.dept_id = d.dept_id;
```

Output:

Error

SQL query: Copy

```
SELECT e.name, e.salary, d.dept_name
FROM Employee e
FULL JOIN Department d ON e.dept_id = d.dept_id LIMIT 0, 25
```

MySQL said:

#1064 - You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'FULL JOIN Department d ON e.dept_id = d.dept_id LIMIT 0, 25' at line 3

Problem 5:

Title: Find employees and salary who work in the IT department.

SQL Query:

```
SELECT e.name, e.salary FROM Employee e JOIN Department d ON e.dept_id = d.dept_id WHERE d.dept_name = 'IT';
```

Output:

Showing rows 0 - 1 (2 total, Query took 0.0005 seconds.)

```
SELECT e.name, e.salary FROM Employee e JOIN Department d ON e.dept_id = d.dept_id WHERE d.dept_name = 'IT';
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

name	salary
Nila	60000
Rina	65000

Problem 6:

Title: Find employees details who work in the department where budget is more than 700000.

SQL Query:
```
SELECT e.name, e.salary, d.dept_name, d.budget FROM Employee e JOIN
Department d ON e.dept_id = d.dept_id WHERE d.budget > 700000;
```

Output:

Showing rows 0 - 1 (2 total, Query took 0.0014 seconds.)

```
SELECT e.name, e.salary, d.dept_name, d.budget FROM Employee e JOIN Department d ON e.dept_id = d.dept_id WHERE d.budget > 700000;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

name	salary	dept_name	budget
Nila	60000	IT	850000
Rina	65000	IT	850000

Problem 7:

Title: Find Highest salary per department.

SQL Query:
```
SELECT d.dept_name, MAX(e.salary) AS highest_salary FROM Employee e JOIN
Department d ON e.dept_id = d.dept_id GROUP BY d.dept_name;
```

Output:

Showing rows 0 - 3 (4 total, Query took 0.0034 seconds.)

```
SELECT d.dept_name, MAX(e.salary) AS highest_salary FROM Employee e JOIN Department d ON e.dept_id = d.dept_id GROUP BY d.dept_name;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

dept_name	highest_salary
Finance	55000
HR	45000
IT	65000
Marketing	*NULL*

Problem 8:

Title: Display Employees whose salary is greater than the average salary

SQL Query
```
SELECT name, salary FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Output:

Showing rows 0 - 1 (2 total, Query took 0.0084 seconds.)

```
SELECT name, salary FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key:

Extra options

				name	salary
Edit	Copy	Delete		Nila	60000
Edit	Copy	Delete		Rina	65000

Problem 9:

Title : Display Employees with their Department Names.

SQL Query:
```
SELECT e.name, d.dept_name FROM Employee e JOIN Department d ON e.dept_id = d.dept_id;
```

Output:

Showing rows 0 - 5 (6 total, Query took 0.0006 seconds.)

```
SELECT e.name, d.dept_name FROM Employee e JOIN Department d ON e.dept_id = d.dept_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: N

Extra options

name	dept_name
Arif	HR
Nila	IT
Sumon	Finance
Rina	IT
Karim	Marketing
Jamil	Finance

Problem 10:

Title: Display Employees who earn more than 60000 and their departments.

SQL Query:
```
SELECT e.name, e.salary, d.dept_name FROM Employee e JOIN Department d ON e.dept_id = d.dept_id WHERE e.salary > 60000;
```

Output:

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

```
SELECT e.name, e.salary, d.dept_name FROM Employee e JOIN Department d ON e.dept_id = d.dept_id WHERE e.salary > 60000;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

name	salary	dept_name
Rina	65000	IT

Problem 11:

Title: Display the Department Name and Total Salary of that department

SQL Query:
```
SELECT d.dept_name, SUM(e.salary) AS total_salary FROM Department d LEFT JOIN Employee e ON d.dept_id = e.dept_id GROUP BY d.dept_name;
```

Output:

Showing rows 0 - 3 (4 total, Query took 0.0016 seconds.)

```
SELECT d.dept_name, SUM(e.salary) AS total_salary FROM Department d LEFT JOIN Employee e ON d.dept_id = e.dept_id GROUP BY d.dept_name;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

dept_name	total_salary
Finance	105000
HR	45000
IT	125000
Marketing	NULL

Result : In this lab session, all SQL queries related to Table JOINs were successfully executed. Using the Employee and Department tables, I was able to generate combined reports that linked employee personal data with their respective department budgets and locations. The use of various JOIN types (INNER, LEFT, RIGHT, FULL) allowed for a flexible way to handle data even when matches were missing.

Discussion : This lab provided a deeper understanding of how to link normalized tables. A major observation was the difference between INNER JOIN and OUTER JOINs; while INNER JOIN only returns matching records, LEFT and RIGHT JOINs are essential for identifying "orphaned" records (e.g., departments with no staff). Another critical learning point was combining JOINs with aggregate functions like SUM and MAX. I observed that using a LEFT JOIN in Problem 11 was better than an INNER JOIN because it allowed the "Marketing" department to appear in the list even though its total salary was NULL.

Conclusion: The lab successfully taught me how to combine information from multiple datasets. I gained practical experience in using table aliases (e.g., e and d) and performing complex joins to answer business questions. These skills are vital for creating real-world database reports where information is split across many tables to maintain data integrity.